

第六模块：固件升级与Flash断电容错（内核级超详细版 · 装备方向）

本模块目标：不是讲“如何烧录程序”，而是讲 长期运行设备如何安全升级且永不变砖。

重点包括： - Flash 底层特性 - 写入原子性 - 双备份升级架构 - 断电恢复机制 - Bootloader 状态机设计

一、Flash 的底层行为（很多人理解错误）

1.1 Flash 不是 RAM

Flash 写入必须遵循：

- 先擦除 (Erase)
- 再写入 (Program)

特点：

- 只能从 1 → 0
- 擦除单位是 Page 或 Sector

工程意义：

不能随意修改一个字节。

1.2 Flash 写入的时间成本

典型耗时：

- 擦除：毫秒级
- 写入：微秒级

期间 CPU 可能被阻塞或必须等待。

实时系统必须考虑：

Flash 操作会影响任务调度。

二、为什么升级会“变砖”（真正原因）

失败根源通常不是代码，而是：

- 写入中断电
- 校验失败
- Boot 状态损坏

最危险阶段：

👉 擦除旧固件但新固件未写完。

三、双备份升级架构（工业级标准做法）

3.1 分区模型

推荐布局：

- Bootloader
- APP_A
- APP_B
- Upgrade Flag 区

核心思想：

始终至少保留一个可启动镜像。

3.2 升级流程（内核级逻辑）

步骤：

1) 下载新固件到备用区 2) 计算 CRC 校验 3) 写入升级标志 4) 重启进入 Bootloader 5) 切换启动地址

如果失败：

自动回滚旧版本。

四、Bootloader 状态机设计（核心能力）

推荐状态：

- BOOT_NORMAL
- BOOT_UPDATE
- BOOT_VERIFY
- BOOT_ROLLBACK

为什么必须状态机：

Flash 写入可能被打断，必须可恢复。

五、断电容错设计（真正底层思维）

5.1 原子标志位（Atomic Flag）

不要用一个变量表示升级完成。

推荐：

- 两阶段提交（Prepare / Commit）

示例：

```
FLAG_PREPARE = 0xAA55  
FLAG_COMMIT  = 0x55AA
```

只有同时满足才认为升级成功。

5.2 日志式写入（Journal 思想）

思想：

不要覆盖旧数据，而是追加记录。

好处：

断电后可以恢复历史状态。

六、Flash 写入与缓存一致性（高级注意）

在有 Cache 的 MCU 上：

- 写入 Flash 后可能需要刷新 I-Cache。
- 否则 CPU 仍执行旧指令。

关键操作：

- Instruction Cache Invalidate
-

七、升级包校验（不仅仅是 CRC）

建议校验层级：

- 长度检查
- CRC 校验
- 版本号检查
- 设备型号检查

装备系统必须防止：

刷错固件。

八、Bootloader 与应用分离原则

Bootloader 必须：

- 尽量小
- 尽量稳定
- 不频繁修改

应用层可以频繁升级。

九、高频底层考点（标准答案）

Q 1：为什么升级必须双备份？

标准答案：

防止升级过程中断电导致系统无法启动。

Q 2：为什么不能边擦边写当前运行区？

标准答案：

一旦写入失败会导致代码区损坏，设备无法运行。

Q 3：Flash 为什么要先擦除？

标准答案：

Flash 物理特性决定只能从 1 写成 0，必须擦除恢复为 1。

Q 4：升级后为什么要刷新 I-Cache？

标准答案：

CPU可能仍执行缓存中的旧指令。

十、工程注意事项（装备级经验）

- 1) Bootloader 不要依赖复杂RTOS。
 - 2) 升级状态必须可恢复。
 - 3) Flash 写入必须关闭中断或使用安全策略。
 - 4) 写入地址必须对齐。
 - 5) 升级日志要可追踪。
-

十一、深度练习（强烈建议）

- 设计一个 APP_A / APP_B 双分区图。
 - 写一个 Boot 状态机流程图。
 - 模拟断电并验证自动回滚逻辑。
-

完成本模块后，你将具备“装备级固件升级架构能力”，这是长期部署设备的核心能力之一。